

Backend Framework: Architectural Deep-Dive

AI-Powered AWS Deployment Assistant • FastAPI, Async Concurrency, AST Deserialization & Service Architecture

This technical reference provides an in-depth breakdown of the backend subsystem powering the **AI-Powered AWS Deployment Assistant**. The backend is engineered in **Python 3.11+** on **FastAPI**, utilizing an asynchronous ASGI architecture to manage I/O-bound LLM orchestrations, strict Pydantic v2 data serialization, and custom CloudFormation AST deserialization.

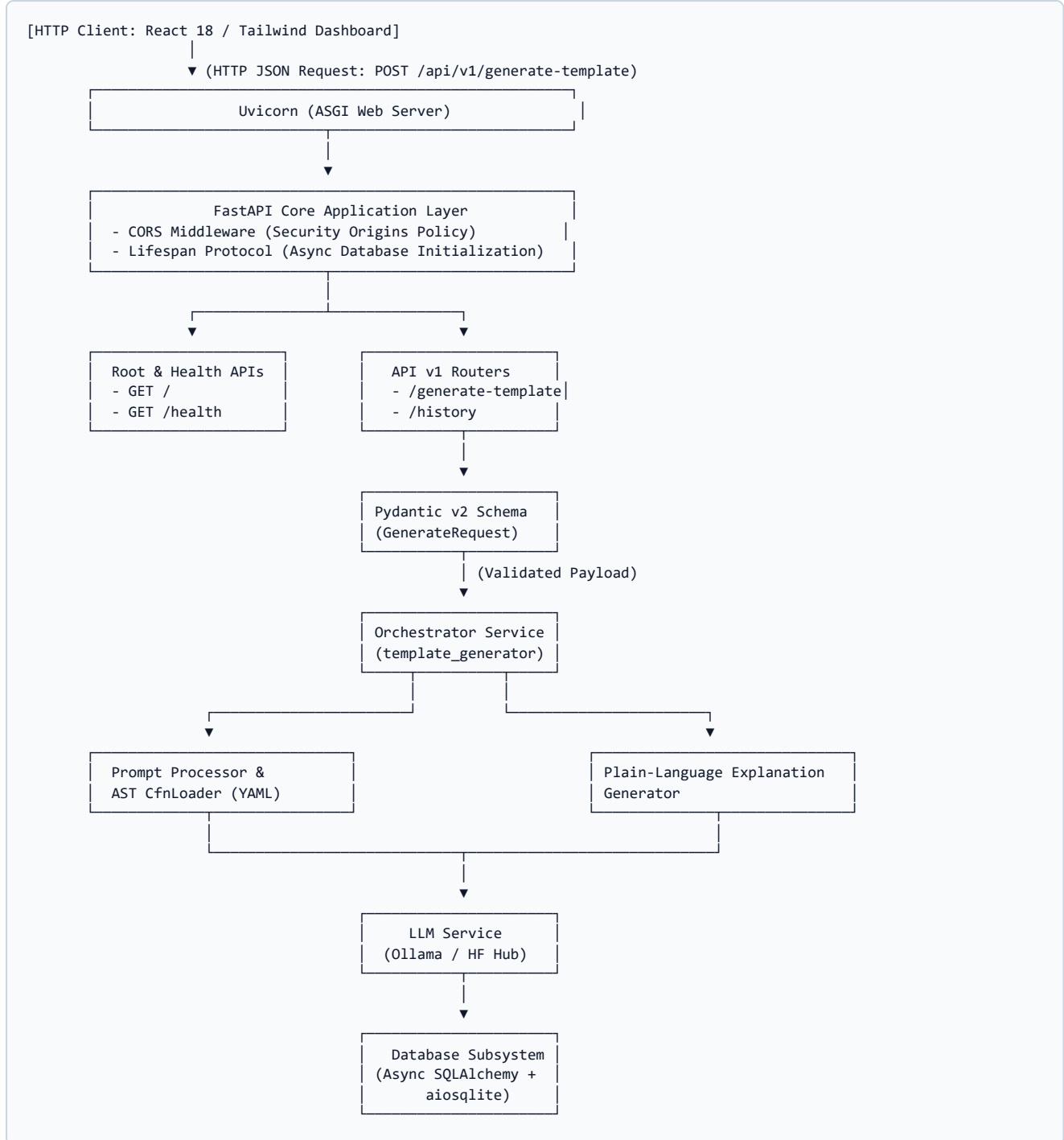
1. Framework Selection: Why FastAPI over Flask or Django?

Modern generative cloud platforms demand low latency, non-blocking I/O during long LLM calls, and guaranteed schema integrity. Below is an architectural comparison:

Dimension	FastAPI (Our Choice)	Flask	Django
Concurrency & Runtime	Native asynchronous ASGI (async/await). Event loop concurrency rivals Go and Node.js.	Synchronous WSGI by default. Async support is retrofitted and incurs worker blocking.	Synchronous WSGI by default. Heavy multi-threaded memory footprint per request.
Schema Validation	Native Pydantic v2 integration with C-based Core (Rust). Zero manual parsing.	Requires manual validation or auxiliary add-ons (Marshmallow, Cerberus).	Django Forms and DRF Serializers (verbose, tightly coupled to relational models).
API Documentation	Automated interactive OpenAPI 3.1 (Swagger UI) at /docs and ReDoc at /redoc.	Requires external third-party extensions (Flask-RESTX).	Requires third-party packages (drf-yasg, drf-spectacular).
Microservice Fit	Lightweight, fast cold-start, modular router structure suited for AI inference microservices.	Lightweight but unopinionated; requires assembling dozens of unaligned modules.	Monolithic "batteries-included" — introduces unnecessary overhead for pure AI APIs.

2. System Architecture & Request Lifecycle

The backend sits between the client application and upstream AI/database layers, maintaining complete separation of concerns:



3. In-Depth Engineering Components

A. ASGI & Asynchronous Non-Blocking Event Loop

Unlike standard web operations that complete in milliseconds, LLM completions take between **5 to 25 seconds**. In synchronous frameworks, incoming HTTP worker threads are completely blocked during generation, causing immediate thread pool starvation.

FastAPI operates on Python's asyncio event loop. When `await generate_with_llm(...)` is called, execution yields control back to the event loop. The server can process concurrent health checks, frontend static assets, and multiple incoming queries seamlessly without blocking.

B. Modern Lifespan Protocol (Database Lifecycle)

Rather than relying on deprecated startup and shutdown hooks, the backend utilizes Starlette's lifespan context manager in `backend/app/main.py`:

```
@asynccontextmanager async def lifespan(app: FastAPI): # Startup Phase: Asynchronously create SQLite tables
    await init_db() yield # Shutdown Phase: Gracefully terminate connection pools
```

This ensures that the database tables (e.g. `prompt_history`) are created asynchronously before the server accepts traffic, preventing race conditions.

C. Inversion of Control: Dependency Injection (`Depends(get\_db)`)

Database sessions are managed using FastAPI's dependency injection system in `backend/app/routers/generate.py`:

```
@router.post("/generate-template", response_model=GenerateResponse) async def generate_template_endpoint(
    request: GenerateRequest, db: AsyncSession = Depends(get_db) ): return await
    generate_template(request.prompt, db)
```

- **Separation of Concerns:** Route handlers remain agnostic to database connection details.
- **Automatic Resource Management:** The session is cleanly opened and guaranteed to be committed, rolled back on errors, and closed in a `finally` block.

D. AST-Level Deserialization & The Custom `CfnLoader`

The CloudFormation Intrinsic Tag Challenge

AWS CloudFormation uses proprietary YAML tags (`!Ref`, `!Sub`, `!GetAtt`, `!Join`). Standard Python parsers (`yaml.safe_load`) immediately fail with `ConstructorError: could not determine a constructor for the tag '!Ref'`.

To solve this, we engineered `CfnLoader(yaml.SafeLoader)` in `backend/app/services/prompt_processor.py`, registering constructors for all 18 CloudFormation tags:

```
CFN_TAGS = [ "!Ref", "!Sub", "!GetAtt", "!Join", "!Select", "!Split", "!FindInMap", "!GetAZs",
"!ImportValue", "!Condition", "!Equals", "!If", "!Not", "!And", "!Or", "!Base64", "!Cidr", "!Transform" ]
def _cfn_tag_constructor(loader, tag_suffix, node): if isinstance(node, yaml.ScalarNode): return
{tag_suffix: loader.construct_scalar(node)} elif isinstance(node, yaml.SequenceNode): return {tag_suffix:
loader.construct_sequence(node, deep=True)} elif isinstance(node, yaml.MappingNode): return {tag_suffix:
loader.construct_mapping(node, deep=True)} return {tag_suffix: None}
```

This converts expressions such as `!Ref WebAppSecurityGroup` into standard AST dictionary nodes (`{"Ref": "WebAppSecurityGroup"}`), enabling seamless downstream processing by cost estimators, security validators, and diagram generators.

E. Layered Clean Architecture

- **Routers Layer:** Pure HTTP request boundary handling status codes and route paths.
- **Schemas Layer:** Pydantic models validating inputs (minimum character lengths) and formatting output contracts.
- **Services Layer:** Business logic, prompt engineering, and LLM calls, decoupled from the web framework.
- **Database Layer:** SQLAlchemy 2.0 async engine with `aiosqlite` driver for lightweight, serverless persistence.